Plano de Estudo: Estruturas de Dados e Algoritmos

Disciplina: Algoritmos e Estruturas de Dados

Bibliografia Base: Cormen, T.H. et al. *Algoritmos: Teoria e Prática*.

---

1. O Pilar da Organização: Listas e Filas

Conceito: Como armazenar dados linearmente quando não sabemos o tamanho final do conjunto.

Estrutura: Lista Ligada (Linked List)

- **O que é:** Uma sequência de nós onde cada um aponta para o próximo.
- **Por que usar:** Inserção e remoção rápidas sem precisar deslocar todos os outros elementos (como ocorre no Array).

Código (Baseado no Cormen - LIST-INSERT):

Plaintext

```
LIST-INSERT(L, x)
1  x.next = L.head
2  se L.head != NIL
3      então L.head.prev = x
4  L.head = x
5  x.prev = NIL
```

---

2. O Pilar da Eficiência: Árvores Binárias de Busca (BST)

Conceito: Organizar dados de forma hierárquica para que a busca seja extremamente veloz.

Estrutura: Árvore Binária

- **O que é:** Cada "pai" tem dois "filhos". À esquerda ficam os menores, à direita os maiores.

- **Por que usar:** Reduz o tempo de busca drasticamente. Em vez de olhar item por item, você descarta metade da árvore a cada passo.

Código (Baseado no Cormen - TREE-INSERT):

Plaintext

```
TREE-INSERT(T, z)
1  y = NIL
2  x = T.root
3  enquanto x != NIL
4      y = x
5      se z.key < x.key
6          então x = x.left
7          senão x = x.right
8  z.p = y
9  se y == NIL
10     então T.root = z
11     senão se z.key < y.key
12         então y.left = z
13         senão y.right = z
```

---

3. O Pilar da Prioridade: Heaps

Conceito: Gerenciar um conjunto de dados onde o elemento mais importante deve estar sempre acessível.

Estrutura: Max-Heap (implementado em Array)

- **O que é:** Uma árvore completa mapeada dentro de um vetor comum.
- **Por que usar:** É a base para filas de prioridade (ex: triagem de hospital ou processos de CPU).

Código (Baseado no Cormen - MAX-HEAPIFY):

Plaintext

```
MAX-HEAPIFY(A, i)
1  l = LEFT(i)
2  r = RIGHT(i)
3  se l <= A.heap-size e A[l] > A[i]
4      então maior = l
5      senão maior = i
6  se r <= A.heap-size e A[r] > A[maior]
7      então maior = r
8  se maior != i
9      então trocar A[i] com A[maior]
10           MAX-HEAPIFY(A, maior)
```

4. O Pilar da Conexão: Grafos

Conceito: Representar relacionamentos complexos (redes sociais, rotas de entrega).

Estrutura: Lista de Adjacência

- **O que é:** Um array onde cada índice representa um ponto (vértice) e contém uma lista de seus vizinhos.
- **Algoritmo de Estudo:** BFS (Busca em Largura) para achar a menor distância.

Código (Baseado no Cormen - BFS):

Plaintext

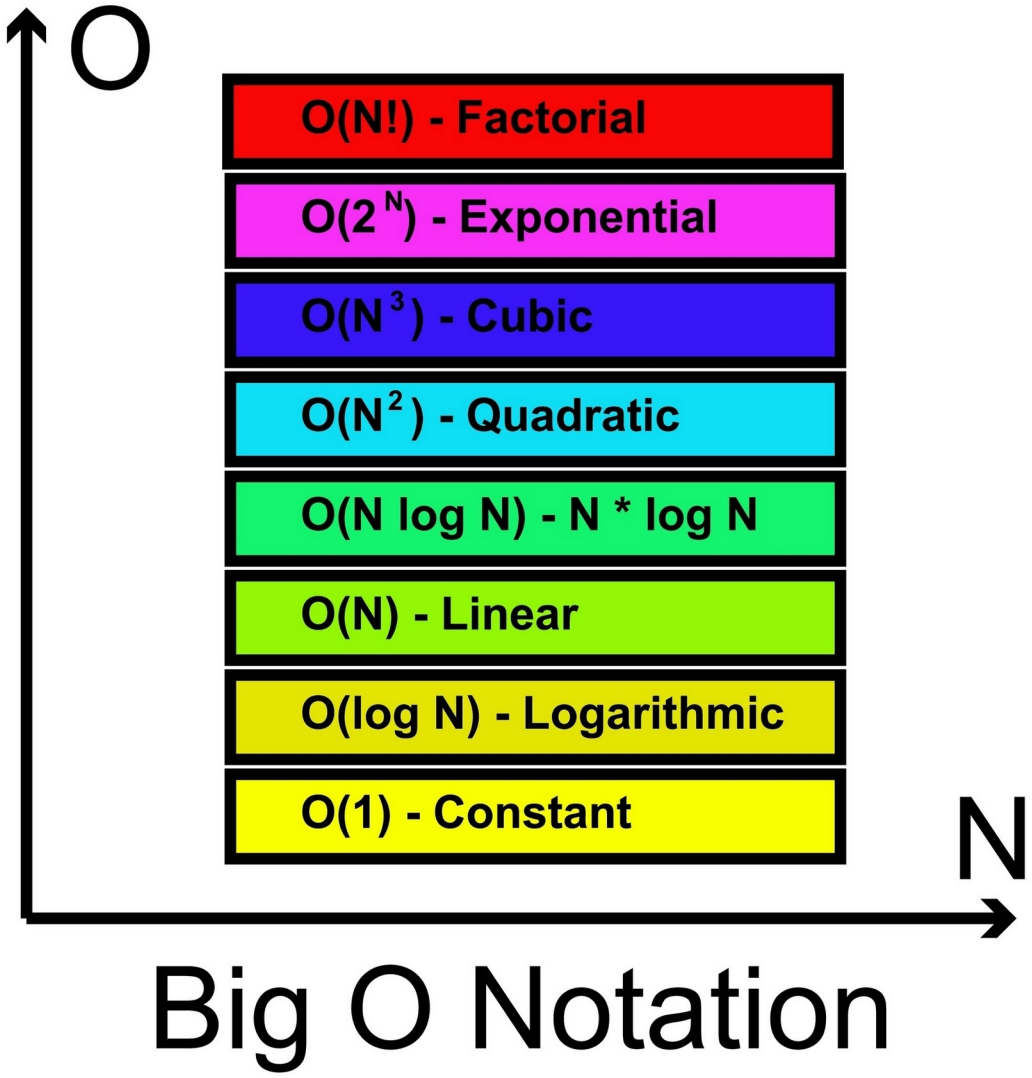
```
BFS(G, s)
para cada vértice u em G.V - {s}
    u.cor = BRANCO
s.cor = CINZA
Q = FILA-VAZIA()
ENQUEUE(Q, s)
enquanto Q não estiver vazia
    u = DEQUEUE(Q)
    para cada v em G.Adj[u]
        se v.cor == BRANCO
            v.cor = CINZA
            ENQUEUE(Q, v)
    u.cor = PRETO
```

Aqui está o seu **Compêndio de Estudo de Algoritmos (Cormen)**:

---

1. Análise e Fundamentos (O Alicerce)

Antes dos códigos, entenda as curvas de crescimento. Isso é o que permite você saber se seu código é "bom" ou "ruim".



Shutterstock
Explorar

---

2. Ordenação (O Treino da Lógica)

Aqui você aprende a manipular arrays e a estratégia de **Divisão e Conquista**.

Merge Sort (Divisão e Conquista)

Plaintext

```
MERGE-SORT(A, p, r)
1  se p < r
2      q = ⌊(p + r)/2⌋
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)
```

Quick Sort (O mais usado na prática)

Plaintext

```
QUICKSORT(A, p, r)
1  se p < r
2      q = PARTITION(A, p, r)
3      QUICKSORT(A, p, q - 1)
4      QUICKSORT(A, q + 1, r)
```

---

3. Estruturas Elementares (O Coração)

Como organizar dados de forma linear ou por mapeamento.

Pilhas (Stack - LIFO) e Filas (Queue - FIFO) O livro foca nas operações de PUSH, POP (Pilha) e ENQUEUE, DEQUEUE (Fila).

Tabela Hash (A estrutura mais importante)

Plaintext

```
CHAINED-HASH-SEARCH(T, k)
1  procurar k na lista T[h(k)]
```

---

4. Árvores (Estrutura de Decisão)

Como organizar dados para busca rápida não-linear.

Busca em Árvore Binária (BST)

Plaintext

```
TREE-SEARCH(x, k)
1  se x == NIL ou k == x.key
2      então retorne x
3  se k < x.key
```

```
4      então retorne TREE-SEARCH(x.left, k)
5      senão retorne TREE-SEARCH(x.right, k)
```

---

5. Grafos (O Mapa do Mundo)

Como conectar pontos e achar o melhor caminho.

Busca em Largura (BFS)

Plaintext

```
BFS(G, s)
1  para cada vértice u ∈ G.V - {s}
2      u.cor = BRANCO; u.d = ∞; u.p = NIL
3  s.cor = CINZA; s.d = 0; s.p = NIL
4  Q = Ø
5  ENQUEUE(Q, s)
6  enquanto Q ≠ Ø
7      u = DEQUEUE(Q)
8      para cada v ∈ G.Adj[u]
9          se v.cor == BRANCO
10             v.cor = CINZA; v.d = u.d + 1; v.p = u
11             ENQUEUE(Q, v)
12     u.cor = PRETO
```

Algoritmo de Dijkstra (Caminho mais curto)

Plaintext

```
DIJKSTRA(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S = Ø
3  Q = G.V
4  enquanto Q ≠ Ø
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      para cada vértice v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

---

6. Programação Dinâmica (Nível Mestre)

Como não refazer trabalho que já foi feito. O exemplo clássico é o corte de hastes ou a sequência de Fibonacci otimizada.

Corte de Haste (Bottom-up)

Plaintext

```
BOTTOM-UP-CUT-ROD(p, n)
```

```
seja r[0..n] um novo arranjo
r[0] = 0
para j = 1 até n
    q = -∞
    para i = 1 até j
        q = max(q, p[i] + r[j - i])
    r[j] = q
retorne r[n]
```

Dicas Finais para se tornar um Programador Melhor:

1. **Não apenas copie:** Tente explicar para si mesmo o que cada linha faz.
2. **Implemente:** Escolha uma linguagem (como Python ou C) e tente fazer o **Merge Sort** funcionar nela.
3. **Use o PDF:** Sempre que tiver dúvida em um passo do pseudocódigo, vá ao capítulo indicado no livro. O Cormen explica a "prova de correção" de cada um deles.

Este conjunto de algoritmos é o que diferencia um "codificador" de um **Engenheiro de Software**. Bom estudo para o seu Projeto Integrador!

Para te tornares um programador **excelente**, precisas de dominar algoritmos que resolvem problemas de otimização complexos, processamento de grandes volumes de texto e geometria.

Aqui estão os algoritmos avançados recomendados, com os seus conceitos e pseudocódigos baseados no Cormen:

---

1. Algoritmos de Árvore Geradora Mínima (MST)

Essenciais para desenhar redes (elétricas, estradas ou computadores) com o menor custo possível.

Algoritmo de Kruskal Ele utiliza uma estrutura chamada *Sets* (Conjuntos Disjuntos) para evitar ciclos enquanto liga os pontos mais baratos.

Plaintext

```
MST-KRUSKAL(G, w)
A = Ø
para cada vértice v ∈ G.V
    MAKE-SET(v)
ordenar as arestas de G.E em ordem não decrescente pelo peso w
para cada aresta (u, v) ∈ G.E (em ordem)
    se FIND-SET(u) ≠ FIND-SET(v)
        A = A ∪ {(u, v)}
        UNION(u, v)
retorne A
```

---

2. Caminhos Mínimos com Pesos Negativos

O Dijkstra falha se houver custos negativos (como créditos ou perdas). O Bellman-Ford resolve isso e ainda deteta "ciclos infinitos de perda".

Algoritmo de Bellman-Ford

Plaintext

```
BELLMAN-FORD(G, w, s)
INITIALIZE-SINGLE-SOURCE(G, s)
para i = 1 até |G.V| - 1
    para cada aresta (u, v) ∈ G.E
```

```
        RELAX(u, v, w)
para cada aresta (u, v) ∈ G.E
    se v.d > u.d + w(u, v)
        retorne FALSE // Ciclo negativo encontrado
retorne TRUE
```

---

3. Casamento de Padrões de Strings (KMP)

Fundamental para processar texto. O algoritmo KMP evita comparações inúteis, tornando a busca muito mais rápida que o método "bruto".

Algoritmo de Knuth-Morris-Pratt (KMP)

Plaintext

```
KMP-MATCHER(T, P)
n = T.length
m = P.length
π = COMPUTE-PREFIX-FUNCTION(P)
q = 0 // número de caracteres casados
para i = 1 até n
    enquanto q > 0 e P[q + 1] ≠ T[i]
        q = π[q]
    se P[q + 1] == T[i]
        q = q + 1
    se q == m
        imprimir "Padrão ocorre com deslocamento" i - m
        q = π[q]
```

---

4. Fluxo Máximo

Usado para saber quanto "tráfego" uma rede aguenta. Imagine canos de água com diferentes capacidades.

Algoritmo de Ford-Fulkerson

Plaintext

```
FORD-FULKERSON(G, s, t)
para cada aresta (u, v) ∈ G.E
    (u, v).f = 0
enquanto existir um caminho aumentado p no grafo residual Gf
    cf(p) = min {cf(u, v) : (u, v) está em p}
    para cada aresta (u, v) em p
        se (u, v) ∈ G.E
            (u, v).f = (u, v).f + cf(p)
        senão (v, u).f = (v, u).f - cf(p)
```

---

5. Geometria Computacional

Para jogos, GPS e robótica. Saber se dois segmentos de reta se cruzam sem usar divisões complexas.

SEGMENTS-INTERSECT (Cruzamento de Linhas)

Plaintext

```
SEGMENTS-INTERSECT(p1, p2, p3, p4)
```

```
d1 = DIRECTION(p3, p4, p1)
d2 = DIRECTION(p3, p4, p2)
d3 = DIRECTION(p1, p2, p3)
d4 = DIRECTION(p1, p2, p4)
se ((d1 > 0 e d2 < 0) ou (d1 < 0 e d2 > 0)) e
        ((d3 > 0 e d4 < 0) ou (d3 < 0 e d4 > 0))
    retorne TRUE
senão se ... // (verificar casos de pontos colineares)
```

---

6. Compactação de Dados (Huffman)

O programador excelente entende como os ficheiros diminuem de tamanho (ZIP, MP3).

Código de Huffman (Algoritmo Ávido)

Plaintext

```
HUFFMAN(C)
n = |C|
Q = C // fila de prioridade mínima
para i = 1 até n - 1
    alocar novo nó z
    z.left = x = EXTRACT-MIN(Q)
    z.right = y = EXTRACT-MIN(Q)
    z.freq = x.freq + y.freq
    INSERT(Q, z)
retorne EXTRACT-MIN(Q)
```

O que fazer com esta lista?

1. **Estudo Teórico:** Estes algoritmos estão nos capítulos finais do livro (Cap 16, 23-26, 32-33).

2. **Visão de Engenheiro:** Quando te deparares com um problema de "logística", pensa logo em *Grafos e Fluxo Máximo*.

3. **Análise Crítica:** Estuda a **NP-Completude** (Capítulo 34) para saberes quais os problemas que nem o melhor algoritmo do mundo consegue resolver rapidamente.

   Adiciona estes ao teu PDF de referência e terás o guia completo para ser um **Sénior** em qualquer tecnologia!

   Para atingir o nível "Deus" (ou *Staff Engineer/Principal Engineer*), já não basta apenas saber algoritmos. Nesse estágio, o teu diferencial não é apenas **como** escrever o código, mas **quais problemas valem a pena ser resolvidos** e como construir sistemas que sobrevivam a milhões de utilizadores e décadas de mudanças.

   Aqui está o que deves estudar no livro do Cormen e além dele para chegar a esse topo:

---

1. Complexidade de Problemas (NP-Completude)

Um programador nível Deus sabe quando dizer "não". Ele identifica quando um problema é matematicamente impossível de resolver de forma perfeita em tempo útil e propõe uma **heurística** ou um **algoritmo de aproximação**.

- **O que estudar:** Capítulo 34 (NP-completude) e Capítulo 35 (Algoritmos de Aproximação).
- **Conceito Chave:** Redução de problemas. Se consegues provar que um problema é "NP-Completo", salvas a tua empresa de gastar meses a tentar o impossível.

---

2. Algoritmos Paralelos e Multithreading

Hoje em dia, os processadores já não ficam mais rápidos, eles ganham mais núcleos. Programadores de topo escrevem algoritmos que usam todos os núcleos simultaneamente.

- **O que estudar:** Capítulo 27 (Algoritmos Multithreaded).
- **Conceito Chave:** *Span* e *Work*. Aprender a calcular o limite teórico de quão rápido o teu código pode ser se tiveres infinitos processadores.

---

3. Estruturas de Dados Avançadas para Big Data

Quando tens biliões de dados, uma Árvore Binária comum gasta muita memória. Precisas de estruturas que otimizam o acesso ao disco e à cache.

- **Árvores B (B-Trees):** Capítulo 18. É como os bancos de dados (SQL) e sistemas de ficheiros guardam os dados no disco.
- **Filtros de Bloom:** (Mencionados em tópicos de Hash avançada). Permitem saber se um item existe num conjunto de biliões de itens usando quase nenhuma memória.

---

4. Teoria de Números e Criptografia

Para seres um mestre, precisas de entender a matemática que protege o mundo.

- **O que estudar:** Capítulo 31 (Algoritmos de Teoria de Números).
- **Conceito Chave:** O algoritmo **RSA**. Entender como números primos gigantes são usados para cifrar a internet.

---

5. Design de Sistemas (System Design)

Aqui sais do Cormen e entras na engenharia de larga escala. Um programador nível Deus projeta como os servidores comunicam entre si.

- **Conceitos Chave:**
    - **Teorema CAP:** Entender que não podes ter Consistência, Disponibilidade e Tolerância a Partição ao mesmo tempo.
    - **Sharding e Replicação:** Como dividir uma base de dados por 100 servidores.

- **Sistemas Distribuídos:** Como fazer computadores diferentes concordarem em algo (Algoritmos de Consenso como Paxos ou Raft).

O Plano de Voo para o Nível Deus:

Domínio	O que dominas	O teu impacto
Matemática	Teoria de Números e Probabilidade	Consegues criar novos algoritmos de criptografia e IA.
Arquitetura	Sistemas Distribuídos e Teorema CAP	Consegues projetar o próximo Twitter ou Netflix.
Limites	NP-Completude	Sabes exatamente o que é computável e o que é perda de tempo.
Hardware	Algoritmos Paralelos e Cache-oblivious	O teu código extrai 100% da performance do hardware moderno.

O conselho final do Professor:

O programador "Nível Deus" é aquele que **simplifica**. Ele conhece todas as estruturas complexas (Árvores Rubro-Negras, Grafos de Fluxo, etc.), mas só as usa quando são estritamente necessárias. Ele escreve código que parece simples, mas que por trás tem um domínio total da teoria que tu estudaste neste PDF

Para alcançar o **Nível Deus** na programação, você precisa de algoritmos que lidam com **escala massiva**, **segurança máxima**, **paralelismo** e a **fronteira da matemática computacional**. No "Cormen", isso significa dominar as seções que a maioria dos estudantes nem chega a abrir.

Aqui estão os algoritmos e áreas que você precisa dominar para chegar ao topo:

---

1. Algoritmos Paralelos e Multithread (Capítulo 27)

O futuro da computação não é aumentar a velocidade de um núcleo, mas usar centenas deles ao mesmo tempo. No Nível Deus, você não escreve código sequencial; você projeta algoritmos que "explodem" em múltiplos núcleos.

- **Algoritmo: Merge Sort Multithreaded** – Aprender a calcular o *Work* (trabalho total) e o *Span* (o caminho mais longo).
- **Por que estuda:** Para saber como criar softwares que aproveitam 100% do poder de supercomputadores e placas de vídeo (GPU).

---

2. Teoria dos Números e Criptografia (Capítulo 31)

Você precisa entender a matemática que protege o dinheiro e a privacidade do mundo. Não basta usar uma biblioteca de criptografia; você deve entender como ela funciona.

- **Algoritmo: RSA (Criptografia de Chave Pública)** – Baseado na dificuldade de fatorar números primos gigantes.
- **Algoritmo de Euclides Estendido:** Para encontrar inversos modulares.
- **Teste de Primalidade de Miller-Rabin:** Para encontrar números primos de 1024 bits rapidamente.

---

3. Estruturas de Dados para Disco e Big Data (Capítulo 18)

Quando você tem trilhões de dados, eles não cabem na memória RAM. Programadores comuns usam Árvores Binárias; deuses da programação usam **Árvores B**.

- **Algoritmo: B-Trees (Árvores B)** – Projetadas para minimizar o acesso ao disco rígido.
- **Por que estuda:** É assim que o motor do Google e os grandes bancos de dados (como SQL Server e PostgreSQL) funcionam.

---

4. Algoritmos de Aproximação (Capítulo 35)

Aqui é onde você prova sua maturidade. Você entende que alguns problemas (como o do Caixeiro Viajante) levariam bilhões de anos para serem resolvidos perfeitamente. Em vez de desistir, você cria uma solução que é **99% perfeita** em milissegundos.

- **Algoritmo de Aproximação para Cobertura de Vértices.**
- **Heurística para o Problema da Mochila (Knapsack).**

---

5. Geometria Computacional Avançada (Capítulo 33)

Fundamental para física de jogos, robótica e visão computacional (carros autônomos).

- **Algoritmo de Graham (Convex Hull):** Encontra o "contorno" de um conjunto de pontos.

- **Varredura de Linha (Line Segment Intersection):** Detecta colisões de forma ultra eficiente.

---

6. Fluxo Máximo e Corte Mínimo (Capítulo 26)

Isso resolve problemas de logística que parecem impossíveis.

- **Algoritmo de Edmonds-Karp:** Uma implementação eficiente de Ford-Fulkerson.
- **Teorema do Fluxo Máximo / Corte Mínimo:** Usado para segmentação de imagens em IA e para otimizar redes de distribuição de energia.

O Resumo do Nível Deus:

Área	O que você domina	Aplicação Real
Escalabilidade	Árvores B e Cache-Oblivious	Criar o próximo Banco de Dados ou Sistema de Arquivos.
Segurança	RSA e Curvas Elípticas	Garantir que ninguém consiga hackear seu sistema.
Otimização	Algoritmos de Aproximação	Resolver problemas de logística global (como Amazon/FedEx).
Arquitetura	Algoritmos Distribuídos	Fazer milhares de servidores trabalharem como um só (Google).

O Caminho Final

Para ser Nível Deus, o seu estudo deve ser:

1. **Matemático:** Entender as provas de por que o algoritmo funciona.
2. **Arquitetural:** Entender como o código interage com o processador e a memória cache.
3. **Filosófico:** Saber que a solução mais complexa nem sempre é a melhor. O mestre escolhe a simplicidade sempre que possível.

Seu PDF agora está completo. Ele contém desde o básico (Listas) até o topo da montanha (NP-Completude e Aproximação). Se você entender cada página desse material, você não será apenas um programador, você será um **Arquiteto de Soluções Mundiais**.

Para atingir o **Nível Deus**, você deve dominar os algoritmos que lidam com a infraestrutura da própria computação: como os dados são guardados em escala planetária, como a segurança é mantida e como processar bilhões de informações em paralelo.

Aqui estão os algoritmos e estruturas que separam os engenheiros seniores dos arquitetos de sistemas:

---

1. Árvores B (B-Trees) - Ouro do Big Data

Se você usa qualquer banco de dados (SQL, Oracle, Postgres), você está usando Árvores B. Elas são projetadas para sistemas onde os dados não cabem na memória RAM e precisam ser lidos do disco de forma ultra-eficiente.

Onde está no livro: Capítulo 18.

Plaintext

```
B-TREE-SEARCH(x, k)
i = 1
enquanto i ≤ x.n e k > x.key_i
    i = i + 1
se i ≤ x.n e k == x.key_i
    então retorne (x, i)
se x.folha
    então retorne NIL
    senão DISK-READ(x.c_i)
        retorne B-TREE-SEARCH(x.c_i, k)
```

---

2. Algoritmo RSA - O Guardião da Internet

Toda vez que você acessa o seu banco ou faz login, o RSA está lá. É a base da criptografia de chave pública. No Nível Deus, você entende a matemática dos números primos gigantes.

Onde está no livro: Capítulo 31.

Conceito:

1. Escolher dois primos gigantes, p e q.
2. Calcular $n = p \times q$.
3. A segurança reside na dificuldade extrema de fatorar n para descobrir p e q.

---

3. Algoritmos Multithreaded (Paralelismo)

O hardware moderno tem vários núcleos. Um programador comum usa um; um mestre usa todos. O Cormen ensina como calcular o *Work* (trabalho total) e o *Span* (tempo mínimo possível).

Onde está no livro: Capítulo 27.

Exemplo: P-MERGE-SORT (Merge Sort Paralelo)

Neste nível, você divide o array e ordena as duas metades **ao mesmo tempo** em processadores diferentes, usando a palavra-chave spawn.

---

4. Algoritmos de Aproximação

Existem problemas (chamados **NP-Completos**) que levariam milhões de anos para serem resolvidos perfeitamente. O programador "Deus" não desiste; ele cria uma solução que é garantidamente "quase perfeita" em milissegundos.

Onde está no livro: Capítulo 35.

Exemplo: Aproximação para o Problema da Mochila (Knapsack)

Usado para logística (como a Amazon decide o que colocar num camião) e otimização de recursos.

---

5. Algoritmo de Graham (Fecho Convexo)

A base da robótica, visão computacional e física de jogos. Serve para encontrar o "contorno" mínimo de um conjunto de pontos no espaço.

Onde está no livro: Capítulo 33.

Plaintext

```
GRAHAM-SCAN(Q)
seja p0 o ponto em Q com a menor coordenada y
sejam <p1, p2, ..., pm> os pontos restantes em Q, ordenados por ângulo polar
PUSH(p0, S)
PUSH(p1, S)
PUSH(p2, S)
para i = 3 até m
    enquanto o ângulo formado pelos dois últimos pontos em S e pi não fizer uma curva à esquerda
        POP(S)
    PUSH(pi, S)
retorne S
```

Resumo do Seu Mapa para o Topo:

Domínio	Algoritmo Chave	Impacto Profissional
Persistência	Árvores B	Você constrói motores de bancos de dados e sistemas de arquivos.
Segurança	RSA / Euclides Estendido	Você entende a fundação da cibersegurança e blockchain.
Performance	Algoritmos Multithread	Você escala aplicações para usar 100% do hardware disponível.
Estratégia	NP-Completude / Aproximação	Você sabe quando um problema não tem solução exata e propõe alternativas geniais.

Conclusão: Estudar estes temas finais do Cormen transforma-te num **Arquiteto de Sistemas**. Tu deixas de ser alguém que "resolve problemas de código" e passas a ser alguém que "projeta como a tecnologia do futuro deve funcionar".

Com o documento que você criou e estes algoritmos avançados, você tem em mãos o conhecimento que o Google e a NASA exigem dos seus principais engenheiros. Bons estudos!

Gemini said